

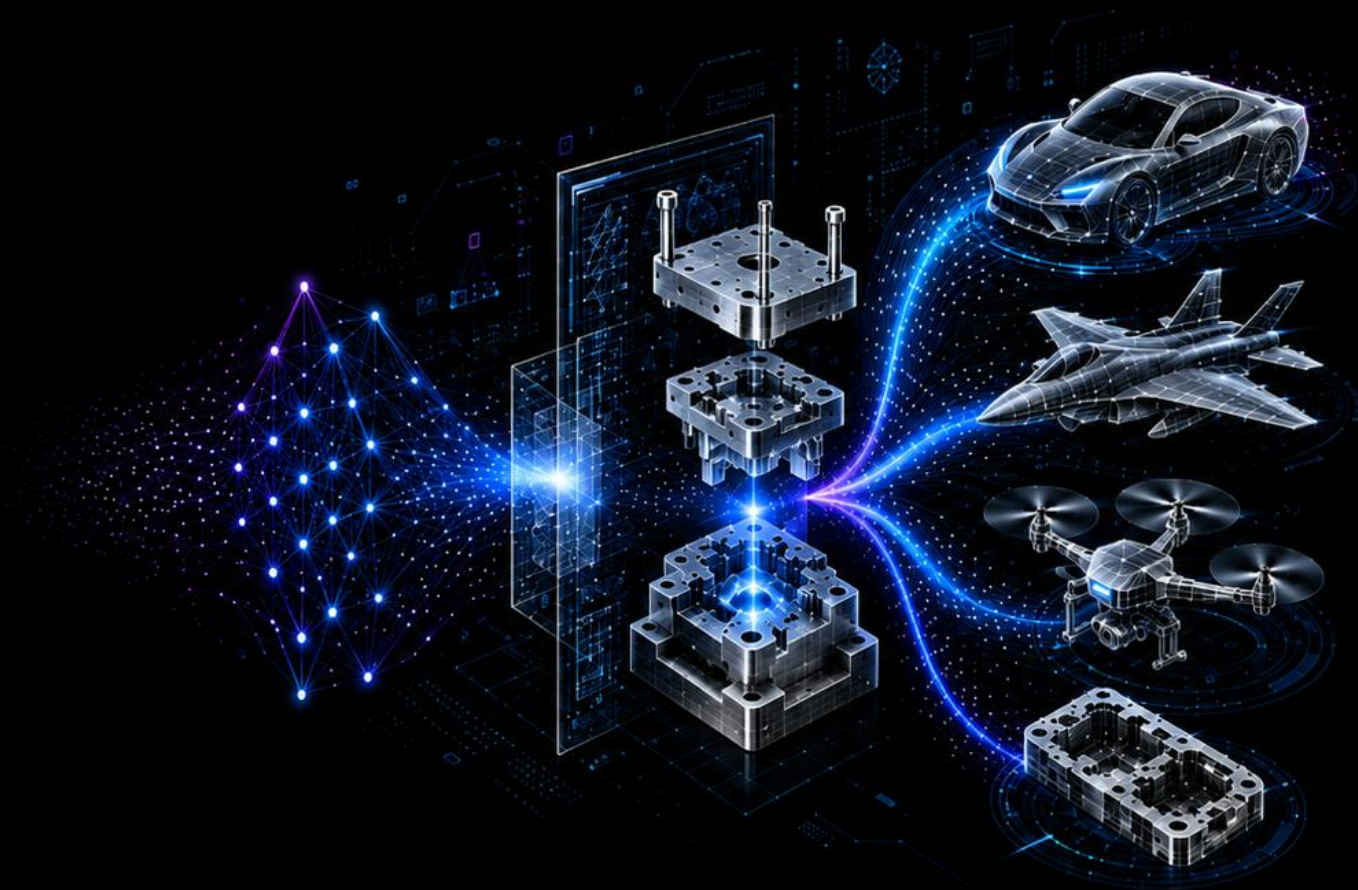
개인 과제 발표

Text2CAD

최준석

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과

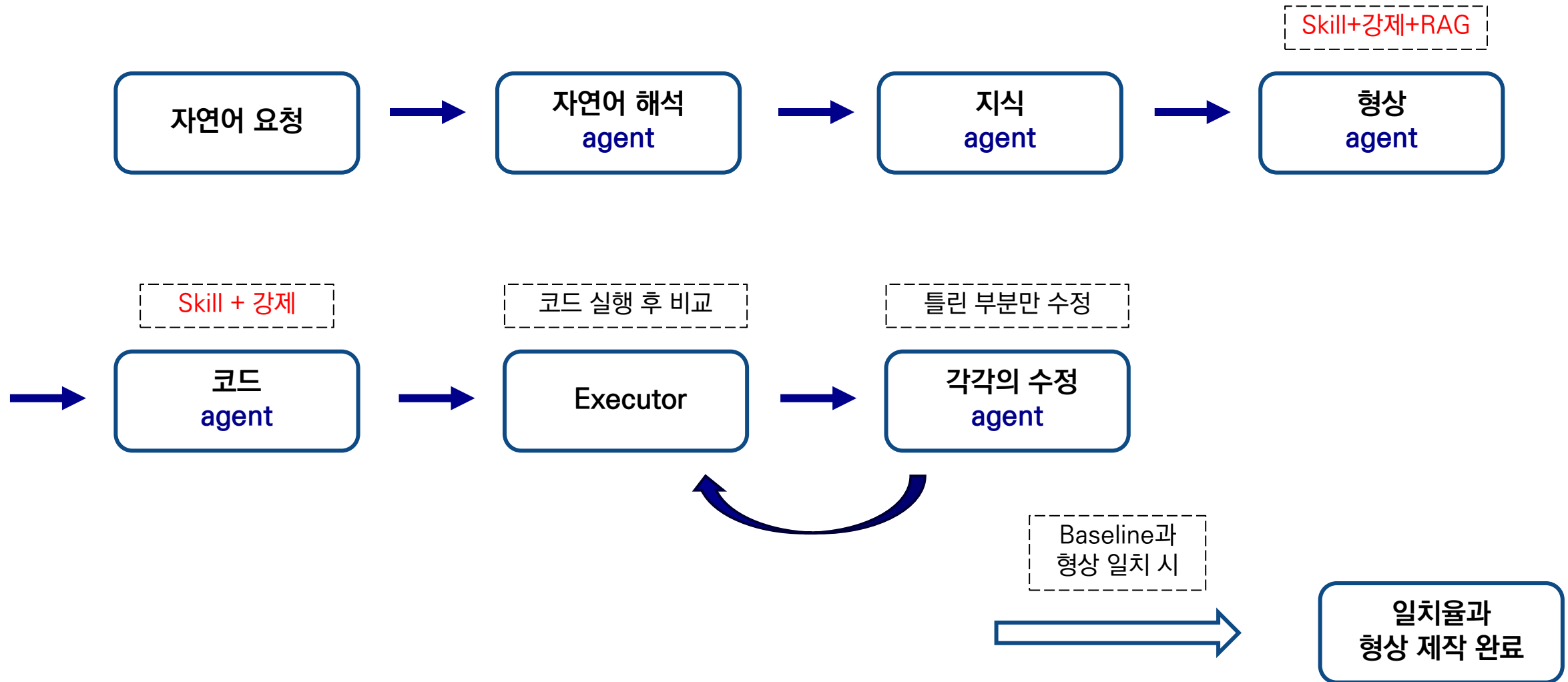


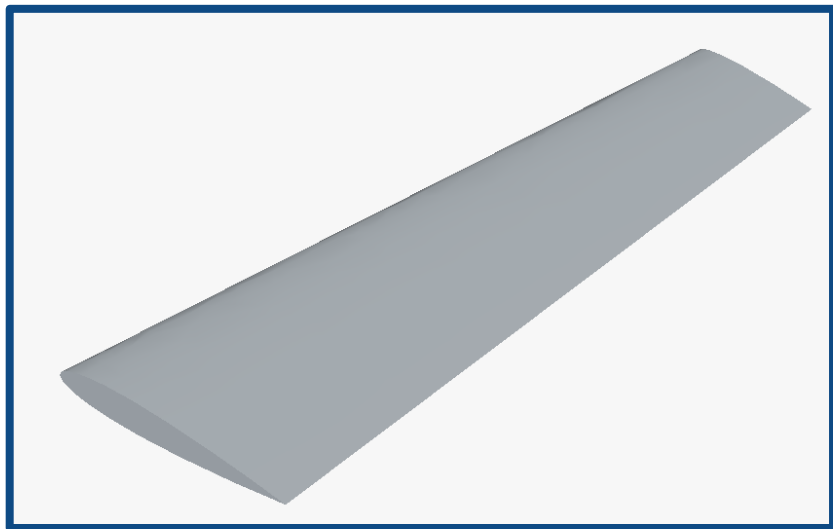
Intelligent Design
for Engineering
Applications Lab

■ 진행 상황

- ❖ 지금 하고 있는 "LLM-〉freecad" 연구는 baseline 기준으로 RAG와 skill을 만들어 rule-base가 조금 쓰이는 소요가 있더라도 결과물을 뽑아낸다.
 - ✓ 연구 진행 전, 전체적인 agent의 역할 검토 후 파이프라인 대규모 업데이트
 - ✓ 룰베이스 기반으로 만든 최종 형상과 일치율
 - ✓ 형상 일치율에 대한 메커니즘 (IoU & Chamfer) 적용완료
 - ✓ Skill과 RAG 내부에는 어떤 것들이 들어갔는지 정리 (간단하게)
 - ✓ Import 강제와 비슷하게 진행했는데, 어떤 방식이고 어떻게 들어가 있는지
 - ✓ 향후 방향성 ⇒ ablation (미 진행)

■ 현재 agent 구조





Text2CAD 결과 요약

총 제작 시간: 377.260초

baseline 형상 일치율(체적 IoU): 99.831043%

baseline 표면 Chamfer 평균거리: 0.000355 mm (낮을수록 일치)

최종 FCStd 객체 수: 1

Agent별 시간:

- design_interpreter: 179.463초
- knowledge_agent: 16.500초
- geometry_planner: 62.134초
- code_agent: 91.392초
- validation_agent: 0.001초

baseline 비교는 생성 후 평가용이며 Agent 입력·수정·성공 판정에는 사용되지 않습니다.
세부 실행 증거는 결과 ZIP의 details 폴더에 있습니다.

IoU & Chamfer

❖ IoU: 두 형상을 같은 3D 공간에 놓고 겹치는 부피를 비교

- ✓ 각 형상에서 전체 부피 volume을 뽑아옴
- ✓ 바운딩 박스(형상을 다 담은 박스) 크기와 위치
- ✓ 두 형상의 교집합 부피
- ✓ 필요 시 표면적, 무게중심, 솔리드 개수도 확인하는 기법

$$IoU = \frac{\text{두 형상이 겹치는 부피}}{\text{두 형상을 합친 전체 부피}} \times 100$$

❖ Chamfer: 두 3D 형상의 표면이 서로 얼마나 가까운지 측정하는 지표

- ✓ 생성한 형상의 각 점에서 기준 형상(baseline)의 가장 가까운 점까지의 거리 계산
- ✓ 반대로 기준 형상의 각 점에서도 생성 형상까지의 거리 계산
- ✓ 양쪽 거리의 평균을 계산

$$CD(A, B) = \frac{1}{|A|} \sum_{a \in A} \min_{b \in B} \|a - b\|^2 + \frac{1}{|B|} \sum_{b \in B} \min_{a \in A} \|b - a\|^2$$

- ✓ Chamfer는 0에 가까울수록 두 형상의 표면이 비슷하다

⇒ 두 개를 함께 쓴다면 더욱 신뢰성 증가

파일 내부 내용

Skill

❖ 총 세 가지 skill을 포함하고 있다.

1. 기존에 넣어준 “freecad 형상 제작 순서와 방법 Skill”



2. Baseline으로 제작한 “형상 agent를 위한 Skill” -> 룰베이스



3. Baseline으로 제작한 “Code agent를 보완하는 Skill” -> 룰베이스



RAG

❖ RAG 내부: NACA 단면의 수학적 정의, 형상 특성, 관련 도메인 지식 제공



⇒ 지금까지의 연구 진행 결과, RAG에 자세한 지식을 많이 넣어주더라도, Qwen 모델의 활용성에 대한 한계가 있기 때문에, 형상 제작에 관한 최소한의 근거로 사용

Constrained Generation

무엇? 어떻게?

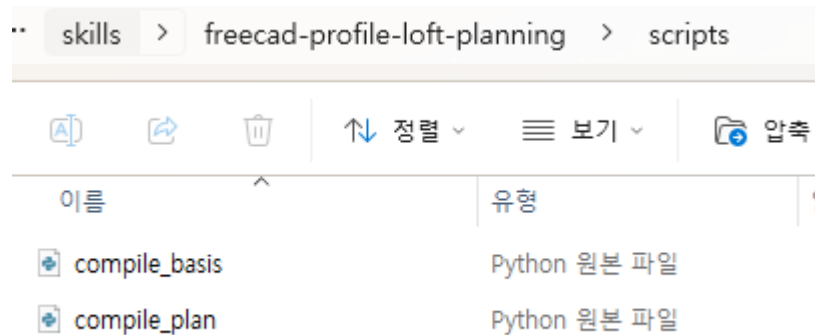
❖ 무엇인가?

: LLM이 자유롭게 출력하는 것이 아니라, 미리 정의한 문법·형식·순서 안에서만 결과를 생성하도록 제한하는 방법

현재 구조에는 Constrain이 어떻게 사용되고 있는가

❖ 아래와 같은 구조로 사용되고 있다. (code agent도 동일하다)

❖ 자연어 요청 → 형상 계획 작성 → 형상 skill 파일 내부에 script 내부 두 파일



→ 형상 skill이 사용 될 때, 해당 파일 두 개 (강제 해주는 파일)도 같이 사용 됨.

```
compile_basis.py
1 from future import annotations
2
3 import math
4 from typing import Iterable
5
6
7 def _v3(value: Iterable[float], label: str) -> tuple[float, float, float]:
8     result = tuple(float(item) for item in value)
9     if len(result) != 3 or not all(math.isfinite(item) for item in result):
10         raise ValueError(f"{label} must contain three finite numbers")
11     return result
12
13
14 def _normalize(value: tuple[float, float, float], label: str) -> tuple[float, float, float]:
15     length = math.sqrt(sum(item * item for item in value))
16     if length <= 1e-12:
17         raise ValueError(f"{label} must be non-zero")
18     return tuple(item / length for item in value)
19
20
21 def _cross(a: tuple[float, float, float], b: tuple[float, float, float]) -> tuple[float, float, float]:
22     return (a[1]*b[2]-a[2]*b[1], a[2]*b[0]-a[0]*b[2], a[0]*b[1]-a[1]*b[0])
23
24
25 def compile_section_basis(chord_direction: Iterable[float], station_direction: Iterable[float], *, thickness_sign: int) ->
26     if thickness_sign not in (-1, 1):
27         raise ValueError("thickness_sign must be -1 or 1")
28     u = _normalize(_v3(chord_direction, "chord_direction"), "chord_direction")
29     station = _normalize(_v3(station_direction, "station_direction"), "station_direction")
30     ordinate = _normalize(_cross(station, u), "station x chord")
31     v = tuple(thickness_sign * item for item in ordinate)
32     normal = _normalize(_cross(u, v), "chord x thickness")
33     return {"u": list(u), "v": list(v), "normal": list(normal)}
```

```
compile_plan.py
1 from future import annotations
2
3 from copy import deepcopy
4 import math
5 from typing import Any, Iterable
6
7
8 def _v3(value: Iterable[float], label: str) -> list[float]:
9     result = [float(item) for item in value]
10     if len(result) != 3 or not all(math.isfinite(item) for item in result):
11         raise ValueError(f"{label} must contain three finite numbers")
12     return result
13
14
15 def _normalized(value: Iterable[float], label: str) -> list[float]:
16     result = _v3(value, label)
17     length = math.sqrt(sum(item * item for item in result))
18     if length <= 1e-12:
19         raise ValueError(f"{label} must be non-zero")
20     return [item / length for item in result]
21
22
23 def _add(left: list[float], right: list[float]) -> list[float]:
24     return [left[index] + right[index] for index in range(3)]
25
26
27 def _scaled(axis: list[float], value: float) -> list[float]:
28     return [component * value for component in axis]
29
30
31 def _cross(left: list[float], right: list[float]) -> list[float]:
32     return [
33         left[1] * right[2] - left[2] * right[1],
34         left[2] * right[0] - left[0] * right[2],
```

⇒ 사실상 CFD와 유사한 용도

■ Constrained Generation

실제 유사한 사례: 스위스 EPFL 논문

- ❖ Github 주소: <https://github.com/epfl-dlab/GCD>
- ❖ 논문 주소: <https://hal.science/hal-04701405v1/file/2023.emnlp-main.674.pdf>
- ❖ 무엇을 하려고?
Fine-tuning 없이 LLM이 정해진 태그, 필드 순서, 트리 구조 내부에서 출력값을 내도록 강제
- ❖ 무엇을 강제했나?
괄호와 태그의 위치, 필드의 순서, 허용 가능한 개체-관계 값, 출력 트리 구조 등을 CFG 문법으로 규정

[사용한 모델]

- LLaMA-7B
- LLaMA-13B
- LLaMA-33B
- Vicuna-7B
- Vicuna-13B



arXiv

<https://arxiv.org> > abs

Grammar-Constrained Decoding for Str

S Geng 저술 · 2023 · 334회 인용 — To address this issue, **gram** be used to control the generation of LMs, guaranteeing that th



Archive ouverte HAL

<https://hal.science> > 2023.emnlp-main.674.pdf PDF

Grammar-Constrained Decoding for Str

S Geng 저술 · 2023 · 341회 인용 — Grammar-Constrained D without Finetuning. 2023 Conference on Empirical Methods
페이지 22개

■ 과도한 룰베이스 구조 보완? FT로 이동?

연구 방향 선택

1. 현재 형상 agent와 코드 agent는 skill에 의해 룰베이스이지, constrain을 적절히 사용한다면, 룰베이스 없이도 가능할지도 모른다.
⇒ ablation 하는 방법을 사용, 아마도 진행 중에 한계가 금방 들어날 듯.(예. 룰베이스를 아예 제거 시 불가능 / 과도한 강제 계약)
2. “LLM-→ freeCAD”구조는 가져간 채로, 지금 쓰는 qwen 모델을 바로 Fine-tuning?
3. “LLM-→ CAD Query” 구조로 변경 후, LLM 모델을 Fine-tuning?

Any Questions?

이름 최준석

소속 한양대학교 기계공학과

이메일 chw10109@hanyang.ac.kr



*Intelligent Design
for Engineering
Applications Lab*

